

GEMINI AI

```
import numpy as np
import matplotlib.pyplot as plt
import random
from datetime import datetime
```

Configuração para gráficos em português

```
plt.rcParams['axes.unicode_minus'] = False
```

def validar_entrada_positiva (mensagem, permitir_zero=False):

```
"""Valida entrada do usuário garantindo que seja um número positivo""" while True:
```

```
    try:
```

```
        valor = float(input(mensagem))
```

```
        if permitir_zero and valor >= 0:
```

```
            return valor
```

```
        elif not permitir_zero and valor > 0:
```

```
            return valor
```

```
        else:
```

```
            print("Erro: Valor inválido. Digite um número positivo.") except
```

```
            ValueError:
```

```
                print("Erro: Digite um número válido.")
```

def validar_percentual (mensagem):

```
"""Valida percentual entre 0 e 100"""
```

```
while True:
```

```
    try:
```

```
        valor = float(input(mensagem))
```

```
        if 0 <= valor <= 100:
```

```
            return valor
```

```
        else:
```

```
            print("Erro: Percentual deve estar entre 0 e 100.")
```

```
    except ValueError:
```

```
        print("Erro: Digite um número válido.")
```

def calcular_indicadores(demanda, perda_percentual, capacidade, volume_inicial,
custo_bombeamento):

```

"""Calcula todos os indicadores para um bairro"""
volume_perdido = demanda * (perda_percentual / 100)
volume_entregue = demanda - volume_perdido
custo_diario = volume_entregue * custo_bombeamento

# Autonomia em dias (considerando demanda real com perda)
if demanda > 0:
    autonomia = volume_inicial / demanda
else:
    autonomia = float('inf')

# Classificação de risco
if autonomia > 3:
    risco = "Baixo"
elif autonomia >= 1:
    risco = "Médio"
else:
    risco = "Alto"

return {
    'volume_entregue': volume_entregue,
    'volume_perdido': volume_perdido,
    'custo_diario': custo_diario,
    'autonomia': autonomia,
    'risco': risco
}

def simular_7_dias(bairros_dados):
    """Simula 7 dias com demanda variando ±20%"""
    n_bairros = len(bairros_dados)

    historico_volumes = {bairro['nome']: [bairro['volume_inicial']] for bairro in bairros_dados}
    dias_criticos = {bairro['nome']: [] for bairro in bairros_dados}

    for dia in range(1, 8):
        for i, bairro in enumerate(bairros_dados):

```

```

# Variação aleatória de ±20% na demanda
demanda_base = bairro['demanda']
variacao = random.uniform(-0.2, 0.2)
demanda_dia = demanda_base * (1 + variacao)

# Volume perdido no dia
volume_perdido = demanda_dia * (bairro['perda'] / 100)
volume_consumido = demanda_dia

# Atualizar volume do reservatório
volume_anterior = historico_volumes[bairro['nome']][-1] novo
volume = volume anterior - volume consumido

#Verificar situação crítica (volume negativo) if
novo_volume < 0:
    dias_criticos[bairro['nome']].append(dia)

# Não permitir volume negativo para continuidade da simulação
novo_volume = 0

historico_volumes[bairro['nome']].append(novo_volume)

# Atualizar dados do bairro para o próximo dia
bairros_dados[i]['volume_inicial'] = novo_volume

return historico_volumes, dias_criticos

def gerar_graficos(bairros_resultados, custos, demandas, volumes_entregues,
historico_volumes=None):
    """Gera todos os gráficos solicitados"""

    nomes = [b['nome'] for b in bairros_resultados]

    # Gráfico 1: Demanda vs Volume Entregue fig, axes =
    plt.subplots(2, 2, figsize=(15, 12))

    x = np.arange(len(nomes))
    width = 0.35

```

```

        axes[0, 0].bar(x - width/2, demandas, width, label='Demanda', color='skyblue') axes[0,
        0].bar(x + width/2, volumes_entregues, width, label='Volume Entregue', color='lightcoral')
axes[0, 0].set_xlabel('Bairros')
axes[0, 0].set_ylabel('Volume (m3)')
axes[0, 0].set_title('Comparação: Demanda vs Volume Entregue')
axes[0, 0].set_xticks(x)
axes[0, 0].set_xticklabels(nomes, rotation=45)
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Gráfico 2: Custo diário por bairro
cores_custo= plt.cm.RdYlGn(np.array(custos) / max(custos))
axes[0, 1].bar(nomes, custos, color=cores_custo)
axes[0, 1].set_xlabel('Bairros')
axes[0, 1].set_ylabel('Custo (R$)')
axes[0, 1].set_title('Custo Diário de Bombeamento')
axes[0, 1].tick_params(axis='x', rotation=45)
axes[0, 1].grid(True, alpha=0.3)

# Gráfico 3: Distribuição dos níveis de risco
riscos = [b['risco'] for b in bairros_resultados]
categorias = ['Baixo', 'Médio', 'Alto']
contagem = [riscos.count(cat) for cat in categorias]
cores_pizza = ['green', 'yellow', 'red']

axes[1, 0].pie(contagem, labels=categorias, autopct="%1.1f%%", colors=cores_pizza,
startangle=90)
axes[1, 0].set_title('Distribuição dos Níveis de Risco')

# Gráfico 4: Evolução dos volumes (simulação 7 dias)
if historico_volumes:
    for nome, volumes in historico_volumes.items():
        dias = list(range(8)) # 0 a 7 dias
axes[1, 1].plot(dias, volumes, marker='o', label=nome, linewidth=2) axes[1,
1].set_xlabel('Dias')
axes[1, 1].set_ylabel('Volume no Reservatório (m3)')

```

```

axes[1, 1].set_title('Evolução do Volume nos Reservatórios (7 dias)') axes[1,
1].legend(loc='best')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

def main():
    """Função principal do programa"""
    print("="*60)
    print("SISTEMA DE ANÁLISE DE ABASTECIMENTO DE
    ÁGUA") print("="*60)

    # Entrada do número de bairros
    while True:
        try:
            n_bairros = int(input("\nNúmero de bairros a analisar: ")) if n_bairros > 0:
                break

            print("Erro: Digite um número positivo.") except
            ValueError:
                print("Erro: Digite um número inteiro válido.")

    bairros =
    demandas = []
    volumes_entregues_lista = []
    custos_lista = []

    # Coleta de dados por bairro
    for i in range(n_bairros):
        print(f"\n--- BAIRRO {i+1} ---")
        nome = input("Nome do bairro: ")

        demanda = validar_entrada_positiva("Demanda diária (m3/dia): ")
        capacidade = validar_entrada_positiva("Capacidade do reservatório (m3): ")
        volume_inicial = validar_entrada_positiva(f"Volume inicial no reservatório (m3) [max:
{capacidade}]: ")

        #Validar volume inicial <= capacidade

```

```

        while volume_inicial > capacidade:
print(f"Erro: Volume inicial não pode exceder a capacidade ({capacidade} m3)") volume_inicial =
        validar_entrada_positiva(f"Volume inicial no reservatório (m3) [max: {capacidade}]: ")
perda = validar_percentual("Percentual de perda na tubulação (%): ") custo_bombeamento =
        validar_entrada_positiva("Custo de bombeamento (R$/m3): ")

        bairros.append({

    })
        'nome': nome,
        'demanda': demanda,
        'capacidade': capacidade,
        'volume_inicial': volume_inicial,
        'perda': perda,
        'custo_bombeamento': custo_bombeamento

# Processamento dos indicadores

print("\n" + "="*60)
print("RESULTADOS DA
ANÁLISE")
print("="*60)

resultados = []

print(f"{'Bairro':<20} {'Vol. Entregue':<15} {'Vol. Perdido':<15} {'Custo (R$)':<15}
{'Autonomia':<12} {'Risco':<8}")
print("-"*85)

for bairro in bairros:
    indicadores calcular_indicadores(
        bairro['demanda'],
        bairro['perda'],

```

```

    )
    bairro['capacidade'],
    bairro['volume_inicial'],
    bairro['custo_bombeamento']

    resultados.append({
        'nome': bairro['nome'],
        **indicadores
    })

    demandas.append(bairro['demanda'])
    volumes_entregues_lista.append(indicadores['volume_entregue'])
    custos_lista.append(indicadores['custo_diario'])

    print(f'{bairro["nome"]: <20} {indicadores["volume_entregue"]:<15.2f}
{indicadores["volume_perdido"]:<15.2f}

        f'{indicadores["custo_diario"]:<15.2f} {indicadores["autonomia"]:<12.2f}
{indicadores["risco"]:<8}')

# Identificação do bairro mais crítico
bairro_critico = min(resultados, key=lambda x: x['autonomia']) print("\n
BAIRRO MAIS CRÍTICO: {bairro_critico['nome']}")
print(f" Autonomia: {bairro_critico['autonomia']:.2f} dias")
print(f" Nível de Risco: {bairro_critico['risco']}")

# Simulação de 7 dias
print("\n" + "="*60)
print("SIMULAÇÃO DE 7 DIAS (Demanda variando
±20%)") print("="*60)

# Criar cópia dos dados para simulação
bairros_simulacao = [b.copy() for b in bairros]
historico_volumes, dias_criticos = simular_7_dias (bairros_simulacao)

```

```

# Exibir dias críticos
for bairro, dias in dias_criticos.items():
    if dias:
        print(f" {bairro}: Dias críticos - {' '.join(map(str, dias))}")
    else:
        print(f"

# Gerar gráficos
        {bairro}: Nenhum dia crítico")

gerar_graficos (resultados, custos_lista, demandas, volumes_entregues_lista,
historico_volumes)

if name
    main__":
    random.seed(42) # Para reprodutibilidade
    main()

```